

Vel Tech Group of Institutions

Name: UMA MAHESHWARI S

Email: vtu28408@veltech.edu.in

Roll no: vtu28408

Phone: null

Branch: VTU

Department: CSE with AIDS

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIDS

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS1L2

VTU_DAA_Week 10_COD

Attempt : 1

Total Mark : 30

Marks Obtained : 22

Section 1 : Coding

1. Problem Statement

Riya, the manager of a software development company, needs to assign three developers to three different projects.

Each developer has a different cost (in terms of time and resource usage) for completing each project. Riya wants to assign exactly one developer to each project such that the total development cost is minimized.

To solve this efficiently, she decides to use the Branch and Bound technique for the Assignment Problem.

Your task is to help Riya determine the minimum total cost of assigning developers to projects.

Input Format

The input consists of three lines with three space-separated integers each, representing a 3×3 cost matrix.

Each cell (i, j) represents the cost of assigning the ith developer to the jth project.

Output Format

The output should display the minimum total cost of the assignment in the following exact format: "Minimum total cost: X", Where X is the minimum total cost obtained after assigning each worker to exactly one task.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 10 12 19

10 5 7

15 14 13

Output: Minimum total cost: 28

Answer

```
#include <stdio.h>
```

```
int cost[3][3];
```

```
int visited[3];
```

```
int minCost = 1000000000;
```

```
void solve(int row, int currCost) {
```

```
    if (row == 3) {
```

```
        if (currCost < minCost)
```

```
            minCost = currCost;
```

```
        return;
```

```
    }
```

```
    for (int j = 0; j < 3; j++) {
```

```
        if (!visited[j]) {
```

```
            visited[j] = 1;
```

```
            solve(row + 1, currCost + cost[row][j]);
```

```
            visited[j] = 0;
```

```

    }
}

int main() {
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            scanf("%d", &cost[i][j]);
        }
    }

    for (int i = 0; i < 3; i++)
        visited[i] = 0;

    solve(0, 0);

    printf("Minimum total cost: %d", minCost);

    return 0;
} // You are using GCC

```

Status : Correct

Marks : 10/10

2. Problem Statement

Arjun is the manager of a disaster relief center responsible for sending essential supplies to a flood-affected area using a single rescue truck. The truck has a limited carrying capacity, so Arjun must carefully select which supply packages to load.

Each package has a specific weight and an importance score representing how critical it is for the affected people. Arjun wants to maximize the total importance score of the packages loaded onto the truck without exceeding its weight capacity.

To solve this efficiently, Arjun uses the Branch and Bound technique, which systematically explores possible combinations of packages while pruning choices that cannot produce better results.

Your task is to help Arjun determine the maximum total importance score

that can be transported.

Input Format

The first line contains an integer N, representing the number of available packages.

The next N lines each contain two integers: weight[i] value[i]

- weight[i] – weight of the package
- value[i] – importance score of the package

The last line contains an integer W, representing the maximum capacity of the rescue truck.

Output Format

The output prints a single integer representing the maximum total importance score that can be carried without exceeding the truck's weight capacity.

Refer to the sample input and output for formatting specifications.

Sample Test Case

Input: 5

2 40

3 50

1 100

5 95

3 30

10

Output: 245

Answer

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
typedef struct {
```

```
    int w, v;
```

```
    double ratio;
```

```
} Item;
```

```
typedef struct {  
    int level;  
    int profit;  
    int weight;  
    double bound;  
} Node;
```

```
int W, N;  
Item arr[1005];
```

```
int cmp(const void *a, const void *b) {  
    Item *x = (Item *)a;  
    Item *y = (Item *)b;  
    if (y->ratio > x->ratio) return 1;  
    return -1;  
}
```

```
// compute upper bound  
double bound(Node u) {  
    if (u.weight >= W) return 0;
```

```
    double profit_bound = u.profit;  
    int j = u.level + 1;  
    int total_weight = u.weight;
```

```
    while (j < N && total_weight + arr[j].w <= W) {  
        total_weight += arr[j].w;  
        profit_bound += arr[j].v;  
        j++;  
    }
```

```
    if (j < N) {  
        profit_bound += (W - total_weight) * arr[j].ratio;  
    }
```

```
    return profit_bound;  
}
```

```
int max(int a, int b) {  
    return a > b ? a : b;  
}
```

```

int main() {
    scanf("%d", &N);

    for (int i = 0; i < N; i++) {
        scanf("%d %d", &arr[i].w, &arr[i].v);
        arr[i].ratio = (double)arr[i].v / arr[i].w;
    }

    scanf("%d", &W);

    qsort(arr, N, sizeof(Item), cmp);

    Node queue[100000];
    int front = 0, rear = 0;

    Node u, v;
    int maxProfit = 0;

    u.level = -1;
    u.profit = 0;
    u.weight = 0;
    u.bound = bound(u);

    queue[rear++] = u;

    while (front < rear) {
        u = queue[front++];

        if (u.bound <= maxProfit || u.level == N - 1)
            continue;

        // take next item
        v.level = u.level + 1;
        v.weight = u.weight + arr[v.level].w;
        v.profit = u.profit + arr[v.level].v;

        if (v.weight <= W && v.profit > maxProfit)
            maxProfit = v.profit;

        v.bound = bound(v);
        if (v.bound > maxProfit)

```

```

        queue[rear++] = v;

        // skip next item
        v.weight = u.weight;
        v.profit = u.profit;
        v.bound = bound(v);

        if (v.bound > maxProfit)
            queue[rear++] = v;
    }

    printf("%d", maxProfit);
    return 0;
} // You are using GCC

```

Status : Correct

Marks : 10/10

3. Problem Statement

You are an art collector who wants to visit a set of art galleries in a city. Each gallery is at a different location, and the cost of traveling between galleries is known.

Your goal is to find the shortest route that allows you to visit each gallery exactly once and return to your starting point, using the traveling salesman problem (TSP). Write a program to calculate the minimum cost of this route.

Note: Solve it using Branch and Bound technique.

Input Format

The first line contains an integer n , the number of galleries.

The next n lines contain n integers each, representing the adjacency matrix.

The i th integer on the j th line represents the cost of traveling from gallery i to gallery j .

Output Format

The output displays "Minimum Cost is: " followed by the minimum cost of the route that visits each gallery exactly once and returns to the starting gallery.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

0 10 15 20

10 0 35 25

15 35 0 30

20 25 30 0

Output: Minimum Cost is: 80

Answer

```
#include <stdio.h>
```

```
#include <limits.h>
```

```
int n;
```

```
int cost[15][15];
```

```
int visited[15];
```

```
int final_res = INT_MAX;
```

```
int firstMin(int i) {
```

```
    int min = INT_MAX;
```

```
    for (int j = 0; j < n; j++) {
```

```
        if (i != j && cost[i][j] < min)
```

```
            min = cost[i][j];
```

```
    }
```

```
    return min;
```

```
}
```

```
int secondMin(int i) {
```

```
    int first = INT_MAX, second = INT_MAX;
```

```
    for (int j = 0; j < n; j++) {
```

```
        if (i == j) continue;
```

```
        if (cost[i][j] <= first) {
```



```

        second = first;
        first = cost[i][j];
    } else if (cost[i][j] <= second) {
        second = cost[i][j];
    }
}
return second;
}

```

```

void TSPRec(int curr_bound, int curr_weight, int level, int curr_path[]) {
    if (level == n) {
        if (cost[curr_path[level - 1]][curr_path[0]] != 0) {
            int res = curr_weight + cost[curr_path[level - 1]][curr_path[0]];
            if (res < final_res)
                final_res = res;
        }
        return;
    }
}

```

```

for (int i = 0; i < n; i++) {
    if (cost[curr_path[level - 1]][i] != 0 && visited[i] == 0) {
        int temp = curr_bound;
        curr_weight += cost[curr_path[level - 1]][i];

        if (level == 1)
            curr_bound -= (firstMin(curr_path[level - 1]) + firstMin(i)) / 2;
        else
            curr_bound -= (secondMin(curr_path[level - 1]) + firstMin(i)) / 2;

        if (curr_bound + curr_weight < final_res) {
            curr_path[level] = i;
            visited[i] = 1;

            TSPRec(curr_bound, curr_weight, level + 1, curr_path);
        }

        curr_weight -= cost[curr_path[level - 1]][i];
        curr_bound = temp;
    }

    for (int j = 0; j < n; j++) {
        visited[j] = 0;
    }
}

```

```

        for (int j = 0; j < level; j++) {
            visited[curr_path[j]] = 1;
        }
    }
}

void TSP() {
    int curr_path[15];
    int curr_bound = 0;

    for (int i = 0; i < n; i++)
        visited[i] = 0;

    for (int i = 0; i < n; i++)
        curr_bound += (firstMin(i) + secondMin(i));

    curr_bound = (curr_bound % 2 == 1) ? curr_bound / 2 + 1 : curr_bound / 2;

    visited[0] = 1;
    curr_path[0] = 0;

    TSPRec(curr_bound, 0, 1, curr_path);
}

int main() {
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &cost[i][j]);
        }
    }

    TSP();

    printf("Minimum Cost is: %d", final_res);
    return 0;
} // You are using GCC

```

Status : Partially correct

Marks : 2/10